

Insertion Sort

Insertion sort builds a sorted prefix of the array one element at a time.

How It Works (Natural Explanation)

Think of sorting playing cards in your hand: - Keep the left side sorted. - Take the next card. - Slide larger cards right until the correct spot opens. - Insert the card.

After round i , the first $i + 1$ elements are sorted.

Formal View

For each index $i = 1, 2, \dots, n - 1$: - Let the current key be $A[i]$. - Shift elements in $A[0..i - 1]$ that are greater than the key one step right. - Place the key into the gap.

Loop invariant: - Before round i , subarray $A[0..i - 1]$ is sorted. - After round i , subarray $A[0..i]$ is sorted.

Why The Time Complexity Is What It Is

- Best case: $O(n)$
 - If the array is already sorted, each round does one comparison and almost no shifting.
- Worst case: $O(n^2)$
 - If the array is reverse sorted, round i shifts about i elements.
 - Total work is:
$$1 + 2 + \dots + (n - 1) = \Theta(n^2).$$
- Average case: $O(n^2)$
 - Typical input still causes a quadratic number of comparisons/shifts overall.

So insertion sort is linear only on nearly-sorted input, but quadratic in general.

When To Use It

- Small arrays.
- Nearly sorted data.
- As a base case inside faster recursive sorts.

Strengths And Limitations

- Strengths:
 - Simple and in-place.
 - Stable (equal elements keep relative order).
 - Very good constant factors for small n .
- Limitations:
 - Poor scaling on large, unsorted data due to $O(n^2)$ behavior.

Related

- [[Merge Sort]]
- [[Divide and Conquer]]
- [[Sorting Algorithms]]
- [[Lectures/Week 2]]